

Implementation of a CAN/CAN-FD Bus Transmitter in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

Abstract	2
Abbreviations	2
1 Introduction	3
1.1 Motivation	3
1.2 Problem Statement	3
1.3 Objectives	3
2 Background	3
2.1 CAN Protocol Evolution	3
2.2 ISO 11898-1:2024 Standard	3
2.3 VHDL and OSVVM	4
3 Verification Planning	4
3.1 Overview and Scope	4
3.2 Requirement Taxonomy	4
3.3 Observability Classification	5
3.3.1 Canonical Layer Interfaces	5
3.3.2 Classification Rules	6
3.4 Verification Plan Construction	6
3.4.1 AI-Assisted Extraction	6
3.4.2 Human-in-the-Loop Validation	7
3.5 Storage Format and Tooling	7
3.6 Verification Strategy	8
4 Design and Architecture	9
4.1 System Overview	9
4.1.1 LLC Frame Format (User Stream and LLC->MAC Stream)	9
4.2 Interface Definition Tables	9
4.3 LLC Sub-layer	17
4.3.1 can_llc_tx	17
4.3.2 can_llc_rx	17
4.4 MAC Sub-layer	17
4.4.1 can_mac_tx	17
4.4.2 can_mac_rx	17
4.5 PCS Sub-layer	17
4.5.1 can_pcs_tx	17
4.5.2 can_pcs_rx	17
5 Implementation	17

5.1	Type Safety and Packages	17
5.2	Bit Timing and TDC	17
5.3	CRC and Bit Stuffing	17
6	Verification and Results	17
6.1	Testbench Results Summary	17
7	Discussion	18
8	Conclusion	18
9	References	18

Abstract

*This document is a work-in-progress draft for the B.Eng thesis. Currently, the project is in the **Verification Plan** phase (Phase 2), with architectural prototyping for the transmitter sub-system completed.*

This thesis describes the design, implementation, and verification of a CAN (Controller Area Network) and CAN-FD (Flexible Data rate) transmitter sub-system. The implementation is targeting high-reliability engine controller applications and is compliant with the ISO 11898-1:2024 standard. Key features include support for both Classic and FD frame formats, Transmitter Delay Compensation (TDC) for high-speed data phases, and a modular architecture separated into Link Layer Control (LLC), Media Access Control (MAC), and Physical Coding Sublayer (PCS).

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgement
AF	Acceptance Field
AUI	Attachment Unit Interface
CAN	Controller Area Network
CBFF	Classical Base Frame Format
CEFF	Classical Extended Frame Format
DF	Data Frame
DLL	Data Link Layer
EF	Error Frame
FBFF	FD Base Frame Format
FCE	Fault Confinement Entity
FEFF	FD Extended Frame Format
FTYP	Frame Type
LLC	Logical Link Control
LSDU	LLC Service Data Unit
MAC	Medium Access Control
OF	Overload Frame
OSI	Open Systems Interconnection

Abbreviation	Meaning
PCS	Physical Coding Sublayer
PDU	Protocol Data Unit
PMA	Physical Medium Attachment
RF	Remote Frame
SAP	Service Access Point
SDU	Service Data Unit
SOF	Start of Frame
TEC/REC	Transmit Error Counter / Receive Error Counter
VCID	Virtual CAN Channel Identifier

1 Introduction

1.1 Motivation

The Controller Area Network (CAN) has been the workhorse of automotive and industrial communication for decades. However, the increasing bandwidth requirements of modern systems led to the development of CAN-FD (Flexible Data rate), which allows for larger payloads and higher bit rates. This project aims to provide a robust, hardware-independent VHDL implementation of a CAN-FD transmitter.

1.2 Problem Statement

Existing CAN implementations often lack flexibility or do not fully support the latest ISO 11898-1:2024 features, such as advanced Transmitter Delay Compensation (TDC). The challenge lies in creating a transmitter that can seamlessly switch between bit rates while maintaining strict protocol compliance and timing accuracy.

1.3 Objectives

- Implement a VHDL-2008 compliant CAN/CAN-FD transmitter.
 - Support both Base (11-bit) and Extended (29-bit) identifiers.
 - Implement TDC measurement and compensation logic.
 - Ensure high verification coverage using OSVVM and GHDL.
-

2 Background

2.1 CAN Protocol Evolution

Brief history from CAN 2.0 to CAN-FD.

2.2 ISO 11898-1:2024 Standard

Overview of the data link layer and physical signaling requirements.

2.3 VHDL and OSVVM

The role of modern VHDL standards and verification frameworks in digital design.

3 Verification Planning

3.1 Overview and Scope

Protocol compliance is the central objective of this project. The CAN and CAN-FD standards (ISO 11898-1:2024) specify hundreds of normative requirements governing frame structure, bit timing, error handling, and fault confinement. To verify the transmitter against these requirements systematically, a structured **Verification Plan** was developed as the first major deliverable of the project - before any RTL implementation began.

The plan covers the four in-scope frame formats: Classic Basic (CB), Classic Extended (CE), FD Basic (FB), and FD Extended (FE). CAN XL frames are excluded. In total, the plan contains 168 requirements extracted from the standard, organized by architectural layer (LLC, MAC, PCS, FCE) and spanning the major functional areas:

- **Frame structure:** Field ordering, bit-level encoding, and format-specific control bits.
- **CRC generation:** CRC-15 (Classic), CRC-17, and CRC-21 (FD) polynomials.
- **Bit stuffing:** Dynamic stuffing during arbitration/data and fixed stuffing in the FD CRC region.
- **Error handling:** Bit error, ACK error, Form error, and Stuff error detection.
- **Fault confinement:** TEC/REC counter management and Error Active/Passive/Bus Off state transitions.

3.2 Requirement Taxonomy

A key design decision was the development of a two-axis taxonomy to classify each requirement by its **shape** and **scope**. This taxonomy determines both *how* a requirement is verified and *what environment* is needed.

Shape classifies the verification primitive. These categories draw on established concepts from formal verification theory:

- **Triggered:** A precondition/event/postcondition triplet that translates directly into a directed test procedure - establish the precondition, apply the event, assert the postcondition. This structure follows the Hoare triple formalism $\{P\} S \{Q\}$ [1].
- **Invariant:** A property that must hold at all times (e.g., “the SOF bit shall always be dominant”), mapping to concurrent assertions or monitors.
- **Liveness:** A property asserting that something eventually happens (e.g., “after detecting an error, the node shall eventually transmit an error flag”), requiring temporal reasoning [2].
- **Reachability:** A property asserting that a state or condition *can* be reached (e.g., “the node shall be capable of entering Bus Off”), mapping to coverage points.

Scope defines the required verification environment:

- **Frame:** Verified by inspecting a single transmitted bit-stream in isolation.
- **Node:** Requires visibility into internal state such as error counters or FSM transitions.

- **Bus:** Requires a multi-node simulation with arbitration and acknowledge behavior.

Together, these two axes allow each requirement to be mapped to the appropriate testbench level and verification technique without ambiguity.

3.3 Observability Classification

The shape and scope taxonomy determines *how* and *where* a requirement is tested, but it does not answer a more fundamental question: *can* the requirement be tested at all from outside the design under test? Not every normative requirement produces a visible effect at the module's ports. Some requirements constrain internal counter thresholds, define structural concepts, or restrict valid configurations - none of which produce a unique, externally distinguishable output. To distinguish these cases systematically, a third classification axis - **observability** - was introduced.

Observability is assessed **per layer**, not per top-level CAN node. The design under test for a PCS requirement is the PCS module in isolation; for a MAC requirement it is the MAC module. This layer-relative framing is essential because a signal that is internal within a full node may be perfectly visible at a sub-layer boundary. For example, the sample point strobe is internal to the CAN node as a whole, but it is the defining output of the PCS module - it determines *when* `PCS_Data.Indicate` fires at the MAC↔PCS boundary and is therefore directly observable in a PCS-level testbench.

3.3.1 Canonical Layer Interfaces

To make observability judgments repeatable and independent of VHDL implementation choices, the classification is anchored in the **canonical service primitives** defined by ISO 11898-1:2015. The standard specifies inter-layer boundaries as abstract service access points with named primitives and parameters:

- **LLC ↔ User:** `L_Data.Request`, `L_Data.Confirm`, `L_Data.Indication` - carrying frame content, transfer status, and timestamps (§6.4.5).
- **MAC ↔ PCS:** `PCS_Data.Request(Output_Unit)`, `PCS_Data.Indicate(Input_Unit)` - the bit-level transmission and reception interface; `PCS_Status.Transmitter(D_Transmit)` and `PCS_Status.Receiver(D_Receive)` - signalling the FD data phase (§7.2).
- **MAC ↔ FCE:** Error notifications (`Error(type)`, `Successful_transfer`), state transition requests (`Error_passive_request`, `Error_active_request`), and responses (`Error_passive_response`, `Error_active_response`) (§8.1.3, Tables 16-17).
- **PCS ↔ FCE:** `Bus_off_request`, `Bus_off_release_request` and their responses (§8.1.3, Tables 18-19).

These canonical interfaces serve as the reference frame for observability: a requirement is classified based on whether its postcondition manifests at the relevant layer's canonical boundary, regardless of how the VHDL implementation names its ports or structures its handshaking.

In addition to the interface primitives, each layer has a set of **configurable values** that the testbench knows at instantiation time - entity generics, constants, or driven stimulus. For the PCS, these include the nominal and FD data bit timing parameters from ISO Table 12 (prescaler, propagation segment, phase segments, SJW) and TDC settings (enable flag, SSP offset). For the MAC, the relevant configurations are error signalling enable and protocol exception enable. For the FCE, there are no user-configurable parameters; its behaviour is entirely determined by fixed counting rules and thresholds defined in §8.1.4.

The combination of canonical interfaces and known configurables provides a complete decision framework:

if a postcondition is fully determined by boundary primitives, configuration generics, and driven stimulus, it is externally verifiable; if it additionally requires knowledge of an internal algorithm, it is derived; if it has no boundary manifestation at all, it is internal.

3.3.2 Classification Rules

From this framework, three classification rules were defined:

Rule 1 - External. A requirement is **external** if its postcondition is fully observable at the layer’s own canonical boundary, in one of two forms. *Rule 1a:* the postcondition maps directly onto a named parameter of a canonical service primitive (e.g., “MAC shall present dominant Output_Unit when transmitting SOF” - Output_Unit is a parameter of PCS_Data.Request). *Rule 1b:* the postcondition manifests as the *timing* of a primitive call, and that timing is completely determined by configuration generics and stimulus inputs known to the testbench (e.g., the sample point position equals $\text{brp} \times (\text{sync_seg} + \text{prop_seg} + \text{phase_seg1})$, all configuration generics, so the testbench can predict and verify exactly when PCS_Data.Indicate fires).

Rule 2 - Derived. A requirement is **derived** if its effect manifests at the layer boundary, but verifying correctness requires knowledge of a **non-trivial internal algorithm** beyond reading configuration generics and measuring timing. The distinction between trivial and non-trivial is important: counting stimulus bits or applying fixed positional offsets is trivial (Rule 1b), while polynomial computation (CRC), state-dependent counter arithmetic (FCE error counters), or multi-step protocol state tracking is non-trivial (Rule 2). For example, CRC bits appear in Output_Unit calls at the MAC↔PCS boundary, but verifying their correctness requires applying the CRC-15, CRC-17, or CRC-21 polynomial to the preceding data - a non-trivial computation that goes beyond what is directly visible at the interface.

Rule 3 - Internal. A requirement is **internal** if its postcondition is a structural definition with no behavioural output (e.g., “the bit time consists of four segments”), a constraint on valid configuration inputs rather than on observable output behaviour (e.g., “Phase_Seg2 shall be $\geq \text{IPT} + \text{SJW}$ ”), or has no manifestation at any layer boundary even indirectly (e.g., oscillator tolerance specifications that are physical constraints not testable in digital simulation).

Applying these rules to the 168 requirements yielded the following distribution: 106 external (63%), 40 derived (24%), and 22 internal (13%). The high proportion of externally observable requirements reflects the standard’s emphasis on bit-level protocol behaviour, which by definition crosses layer boundaries. The internal requirements are concentrated in the PCS layer (configuration constraints and oscillator tolerances) and the FCE (structural definitions of counting concepts). Each requirement’s rationale field records which rule was applied and which canonical interface primitive or configurable value justifies the classification, providing a fully auditable trail from ISO clause to testability assessment.

3.4 Verification Plan Construction

With the taxonomy and observability framework defined, the next step was to extract and classify the normative requirements from the ISO standard text. This was a two-stage process combining AI-assisted extraction with manual engineering review.

3.4.1 AI-Assisted Extraction

The initial extraction of normative “shall” and “should” statements was performed using a **Large Language Model (LLM)**. The ISO standard was provided as a searchable markdown document, and the LLM was

prompted to identify normative clauses, extract their wording verbatim, assign a shape and scope classification, and format the result as TOML entries. This task is well-suited for LLMs because it involves processing large volumes of technical text, identifying structural patterns (normative vs. informative language), and reformatting unstructured prose into a consistent structured format. The use of AI significantly accelerated the initial drafting phase and reduced the risk of human oversight during the translation from standard text to a machine-readable plan.

3.4.2 Human-in-the-Loop Validation

The AI-generated extraction served only as a first draft. Every requirement - its wording, assigned shape, scope, layer, and flags - underwent a comprehensive **manual review**. This step was critical to:

- Verify the technical accuracy of the AI’s interpretation of protocol nuances (e.g., distinguishing between transmitter-side and receiver-side obligations).
- Refine the shape/scope classification where the standard’s wording was ambiguous or where a single clause contained multiple independent requirements (flagged as COMPOUND).
- Identify requirements that depend on components outside the TX pipeline (flagged as EXTERNAL_DEP) or that are advisory rather than mandatory (flagged as SHOULD).
- Ensure that the resulting plan provides a reliable and authoritative basis for the subsequent VHDL implementation and testbench development.

3.5 Storage Format and Tooling

The Verification Plan is stored as a single TOML file (`verification_plan/verification_plan.toml`). TOML was selected for two practical reasons: (1) it is easy to read and edit by hand - each requirement is a self-contained `[[requirement]]` block with one key per line, making version-control diffs clean and merge conflicts rare; and (2) it can be parsed and manipulated programmatically by Python tools (specifically `tomlkit`, which preserves comments and formatting on round-trip). This second property was essential for building the automated tooling described below.

Each requirement entry carries metadata fields designed to drive the verification workflow:

Table 2: Verification-plan metadata fields and their intended use in workflow automation.

Field	Purpose
<code>shape</code>	Classification axis described in Section 3.2 - determines the verification primitive.
<code>scope</code>	Classification axis described in Section 3.2 - determines the test environment.
<code>layer</code>	Architectural sub-layer (LLC, MAC, PCS, or FCE), enabling a divide-and-conquer approach where each testbench targets one layer.
<code>precondition / event / postcondition</code>	For <i>triggered</i> -shape requirements, these three fields form a testable triplet that translates directly into a test procedure.
<code>coverage_target</code>	Describes how to verify the requirement (e.g., “assert CRC field matches polynomial output”, “cover all four frame formats”).

Field	Purpose
observability	Layer-relative testability classification (<i>external</i> , <i>derived</i> , <i>internal</i>) as defined in Section 3.3.
observability_rationale	Auditable justification citing the specific classification rule and canonical interface primitive.
flags	Marks special properties: COMPOUND, AMBIGUOUS, EXTERNAL_DEP, SHOULD, or DOC_ONLY. These flags inform review priority and test generation strategy.
label / file	Link the requirement to its implementing assertion label or testbench procedure, providing full traceability from standard clause to RTL.

To maintain the integrity of the plan as it evolves, a **Model Context Protocol (MCP) server** (`mcp_tools/verification_plan_manager.py`) was developed. The server exposes query, update, insert, delete, and statistics operations as tool calls that the AI coding agent can invoke directly within the development environment. Each write operation creates an automatic backup and validates field values against the schema before committing changes. This design serves two purposes: first, the agent receives summarized query results rather than the entire raw file, avoiding context window bloat; and second, constraining the agent to narrow, validated operations minimizes the risk of data corruption and hallucination - rather than asking the LLM to rewrite a large structured file (where it may silently drop entries, fabricate field values, or break TOML syntax), each MCP call targets a single atomic change with schema-level validation, making such errors structurally impossible.

3.6 Verification Strategy

The verification plan drives a layered testing strategy, where each level targets a different scope of requirements. The observability classification directly informs this strategy: *external* requirements can be verified through port-level stimulus and observation alone, *derived* requirements additionally need a reference model (e.g., a software CRC or error counter model) to compute the expected result, and *internal* requirements are verified through design review or configuration validation rather than simulation.

- **Unit Testing:** Individual modules (Serializer, CRC, Bit Stuffer) are verified in isolation against *frame*-scope requirements. External requirements at this level are verified by driving stimulus and checking output bits; derived requirements (e.g., CRC correctness) use a software reference model for comparison.
- **Protocol Testing:** `tx_can_protocol_tb` verifies frame structure and field timing, covering *node*-scope requirements that involve FSM state and internal counters. Derived FCE requirements (error counter thresholds) are tested by injecting controlled error sequences and observing the resulting state transitions at the MAC↔FCE boundary.
- **Integrated Testing:** `tx_can_tb` verifies end-to-end transmission, retries, and abort scenarios, targeting *bus*-scope requirements that involve multi-node arbitration and acknowledgment.

4 Design and Architecture

4.1 System Overview

A complete CAN node is composed of a Transmit (TX) path and a Receive (RX) path coordinated by shared fault-confinement and physical-interface control. The paths operate in parallel at runtime, but they are not symmetric in behavior or responsibility.

The focus of this thesis is the complete node architecture and its standards-traceable interfaces. The TX path is responsible for frame submission, arbitration participation, and bitstream generation toward the bus, while the RX path is responsible for bus observation, frame reconstruction, and delivery/notification toward the user layer. As shown in Figure 1, this full-node decomposition spans LLC, MAC, PCS, FCE, and PMA boundaries.

4.1.1 LLC Frame Format (User Stream and LLC->MAC Stream)

The canonical LLC stream format is byte oriented with:

- mandatory 6-byte header (CFG0, CFG1, ID3, ID2, ID1, ID0)
- optional payload (0 . . 64 data bytes)

LLC frame packet layout (fixed byte chunks, max FD envelope):

Legend: base-ID formats use ID1 : ID0; extended-ID formats use ID3 : ID0.

b7	b6	b5	b4	b3	b2	b1	b0
FORMAT[2]	FORMAT[1]	FORMAT[0]	FTYP	ESI	BRS	Reserved	Reserved

Table 1: Config Byte 0 bit layout (MSB-left, b7 to b0)

Config Byte 1 bit layout (MSB-left, b7 to b0):

b7	b6	b5	b4	b3	b2	b1	b0
DLC[3]	DLC[2]	DLC[1]	DLC[0]	Reserved	Reserved	Reserved	Reserved

Table 2: Config Byte 1 bit layout (MSB-left, b7 to b0)

Notes:

- sop= '1' on Config Byte 0.
- The first 6 bytes are always present in every LLC frame.
- eop= '1' on final data byte, or on ID0 when DLC=0.

4.2 Interface Definition Tables

The following tables define the interface bundles shown in Figure 1. ISO references are to ISO 11898-1:2024 clauses and tables.

Detailed interface definitions (normative source mapping):

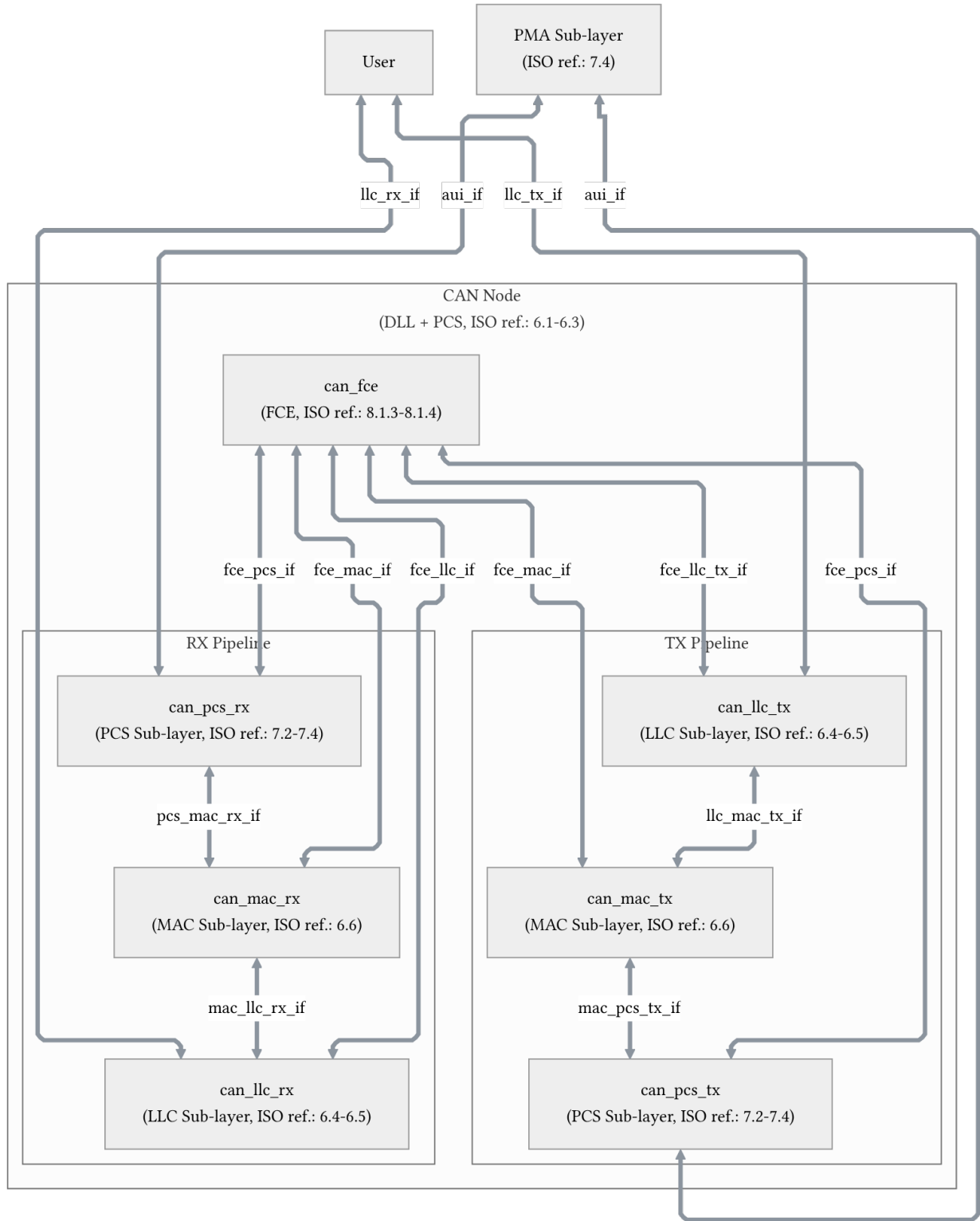


Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table 3), llc_rx_if (Table 4), llc_mac_tx_if (Table 5), mac_llc_rx_if (Table 6), mac_pcs_tx_if (Table 7), pcs_mac_rx_if (Table 8), auif (Table 9), fce_llc_tx_if and fce_llc_if (Table 10), fce_mac_if (Table 11), and fce_pcs_if (Table 12).



Figure 2: LLC frame packet layout (fixed byte chunks, maximum FD envelope).

Table 3: Interface definition for `llc_tx_if`. Uses the existing Avalon-ST interface (`pk_eth_st`) to implement `L_Data.Request`. Includes `L_Data.AbortRequest` for complete ISO-defined service coverage. `L_Data.AbortRequest` is not implemented in the current CAN bus controller. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
<code>L_Data.Request (start)</code>	LLC Frame, Handle	User -> LLC	Request data transfer of LLC frame	Issued when User wants to start LLC frame transfer	6.4.5.5.2	valid && ready && sop && payload on data
<code>L_Data.Request (stream)</code>	LLC Frame, Handle	User -> LLC	Stream LLC frame	Intermediate beats while LLC frame is streamed	6.4.5.5.2	valid && ready && !sop && !eop && payload on data
<code>L_Data.Request (end)</code>	LLC Frame, Handle	User -> LLC	End of current LLC frame transfer	Final beat of LLC frame transfer	6.4.5.5.2	valid && ready && eop && payload on data
<code>L_Data.AbortRequest</code>	Handle	User -> LLC	Abort pending LLC frame transfer	Issued when User wants to cancel LLC frame transfer	6.4.5.5.3	TBD

Table 4: Interface definition for `llc_rx_if`. Uses the existing Avalon-ST interface (`pk_eth_st`) to implement `L_Data.Indication`. Includes `L_Data.Confirm` and optional `Timestamp` for complete ISO service coverage. `L_Data.AbortRequest` is not implemented in the current CAN bus controller. `Transfer_Status`: [Ongoing, Lost Arbitration, Transmitted, Aborted, Disturbed]. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
<code>L_Data.Indication (start)</code>	LLC Frame, optional Timestamp	LLC -> User	Start received LLC frame transfer	Issued when LLC wants to start LLC frame transfer	6.4.5.5.5	valid && ready && sop && payload on data
<code>L_Data.Indication (stream)</code>	LLC Frame, optional Timestamp	LLC -> User	Stream received LLC frame	Intermediate beats while LLC frame is streamed	6.4.5.5.5	valid && ready && !sop && !eop && payload on data
<code>L_Data.Indication (end)</code>	LLC Frame, optional Timestamp	LLC -> User	End received LLC frame transfer	Final beat of LLC frame transfer	6.4.5.5.5	valid && ready && eop && payload on data

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
L_Data.Confirm	Transfer_Status, optional Timestamp, Handle	LLC -> User	Report result of prior L_Data.Request	Issued on completion/failure of prior L_Data.Request	6.4.5.5.4	TBD

Table 5: Interface definition for llc_mac_tx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
DLL SDU	LLC Frame	LLC -> MAC	Transfer LLC Frame content	Issued when LLC starts MAC handoff	6.3, 6.6.4.2	TBD
Interface control information	Control context	LLC -> MAC	Transfer control context for frame handling	Valid with DLL SDU handoff	6.6.4.2	TBD

Table 6: Interface definition for mac_llc_rx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
DLL SDU	Reconstructed LLC Frame	MAC -> LLC	Transfer reconstructed LLC Frame	Issued when MAC has reconstructed LLC Frame from received MAC Frame	6.6.4.3, 6.6.9	TBD
Time reference notification	SOF/frame-valid indication	MAC -> LLC	Provide timing reference for timestamping	Generated for transmitted/received DF/RF	6.6.3	TBD

Table 7: Interface definition for mac_pcs_tx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
PCS_Data.Request	Output_Unit	MAC -> PCS	Request transmission of one dominant/recessive bit	Bit-by-bit during MAC Frame serialization	7.2.1, 7.2.2	TBD
PCS_Status.Transmitter	D_Transmit	MAC -> PCS	Indicate FD data-phase interval	Active during transmission in the FD data-phase	7.2.1, 7.2.5	TBD

Table 8: Interface definition for pcs_mac_rx_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
PCS_Data.Indicate	Input_Unit	PCS -> MAC	Indicate arrival of one dominant/recessive bit	Bit-by-bit during reception stream	7.2.1, 7.2.3	TBD
PCS_Status.Receiver	D_Receive	MAC -> PCS	Indicate FD data-phase interval	Active during reception in the FD data-phase	7.2.1, 7.2.6	TBD

Table 9: Interface definition for au_i_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
output symbol	Dominant/recessive symbol	PCS -> PMA	Drive physical output symbol	Issued on Output_Unit updates	7.4.2.1	TBD
bus_off symbol	Bus-off control	PCS -> PMA	Switch node off bus	Issued on Bus_off_request request from FCE	7.4.2.2, 8.1.3.4	TBD
bus_off_release symbol	Bus-off release control	PCS -> PMA	Release node from bus-off	Issued on Bus_off_release request from FCE	7.4.2.3, 8.1.3.4	TBD

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
input symbol	Dominant/recessive symbol	PMA -> PCS	Indicate physical input symbol	Continuous PMA-to-PCS indication	7.4.3	TBD

Table 10: Interface definition for fce_llc_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Normal_mode_request	Mode request	LLC -> FCE	Request reset to normal mode	Issued on startup/restart	8.1.3.2	TBD
Normal_mode_response	Mode response	FCE -> LLC	Acknowledge normal-mode request	Returned after FCE processing	8.1.3.2	TBD
Bus_off	Bus-off status	FCE -> LLC	Indicate node is bus-off	Asserted on bus-off transition	8.1.3.2	TBD

Table 11: Interface definition for fce_mac_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Transmit/receive	Transfer mode context	MAC -> FCE	Report current TX/RX context	Updated with MAC transfer context	8.1.3.3	TBD
Error	Error event	MAC -> FCE	Report detected protocol error	On bit/stuff/CRC/form/ACK error	8.1.3.3	TBD
Primary_error	Primary error event	MAC -> FCE	Report primary error condition	On primary error condition	8.1.3.3	TBD
Error/overload flag	EF/OF state	MAC -> FCE	Report EF/OF transmission state	During EF/OF transmission	8.1.3.3	TBD
Counters_unchanged	Counter-update qualifier	MAC -> FCE	Qualify counter exception path	On rule-c exception cases	8.1.3.3	TBD

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Error_delimiter_too_late	Late delimiter event	MAC -> FCE	Report late error-delimiter condition	Set on late delimiter condition	8.1.3.3	TBD
Successful_transfer	Transfer completion event	MAC -> FCE	Report successful TX/RX completion	On successful frame transfer	8.1.3.3	TBD
Error_passive_response	State response	MAC -> FCE	Report entry into error-passive state	On state transition completion	8.1.3.3	TBD
Error_active_response	State response	MAC -> FCE	Report return to error-active state	On state transition completion	8.1.3.3	TBD
Error_passive_request	State request	FCE -> MAC	Request MAC enter error-passive state	On TEC/REC threshold crossing	8.1.3.3	TBD
Error_active_request	State request	FCE -> MAC	Request MAC return to error-active state	On TEC/REC recovery conditions	8.1.3.3	TBD

Table 12: Interface definition for fce_pcs_if. TBD = To Be Defined.

ISO symbol	ISO payload	Direction	Semantics	Timing	ISO ref.	Implementation mapping
Bus_off_request	Bus-off request	FCE -> PCS	Request node switch-off from bus	On bus-off transition condition	8.1.3.4	TBD
Bus_off_release_request	Bus-off release request	FCE -> PCS	Request node re-enable from bus-off	On restart/reintegration	8.1.3.4	TBD
Bus_off_response	Bus-off response	PCS -> FCE	Acknowledge bus-off request	Returned after bus-off action	8.1.3.4	TBD
Bus_off_release_response	Bus-off release response	PCS -> FCE	Acknowledge bus-off-release request	Returned after release action	8.1.3.4	TBD

4.3 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

4.3.1 `can_llc_tx`

4.3.2 `can_llc_rx`

4.4 MAC Sub-layer

The core of the protocol logic. It handles bit serialization, CRC calculation, and bit stuffing. It coordinates the overall frame state machine and interacts closely with the Fault Confinement Entity (FCE) to manage error counters (TEC/REC) and node state (Error Active/Passive/Bus Off) based on protocol violations.

4.4.1 `can_mac_tx`

4.4.2 `can_mac_rx`

4.5 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

4.5.1 `can_pcs_tx`

4.5.2 `can_pcs_rx`

5 Implementation

5.1 Type Safety and Packages

The implementation uses custom record types (defined in `can_types_pkg.vhd`) to ensure clean interfaces between modules.

5.2 Bit Timing and TDC

Detailed description of how `tx_pcs` measures propagation delay and calculates the SSP.

5.3 CRC and Bit Stuffing

Implementation details of the flexible CRC generator and the hybrid bit stuffer.

6 Verification and Results

Note: This section is currently being populated as the verification plan is executed.

6.1 Testbench Results Summary

Table 13: Testbench execution status and functional coverage summary.

Testbench	Status	Coverage
tx_pcs_tb	Pass	95%
tx_can_tb	Pass	88%
tx_can_protocol_tb	Pass	92%

7 Discussion

Comparison of the implemented architecture against theoretical models. Performance analysis in high-load scenarios.

8 Conclusion

Summary of work completed and how objectives were met.

9 References

- [1] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, Oct. 1969, doi: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259).
- [2] B. Alpern and F. B. Schneider, “Defining liveness,” *Information Processing Letters*, vol. 21, no. 4, pp. 181–185, Oct. 1985, doi: [10.1016/0020-0190\(85\)90056-0](https://doi.org/10.1016/0020-0190(85)90056-0).